LAB SESSION · DATABASE SYSTEMS LAB

Aggregation, Grouping & Subqueries

Hands-on SQL — write every query, verify every result

Md. Rakib Trofder

Lecturer, Department of CSE
BAUST, Saidpur Cantonment

Saad Ahmed

Lecturer, Department of CSE
BAUST, Saidpur Cantonment



COUNT · SUM · AVG · MIN · MAX · GROUP BY · HAVING · SUBQUERIES

BEFORE YOU START

The Lab Database

Every query in this session runs on these three tables. Load the script first, then keep this slide open — you will verify each result against this data by hand.

STUDENT

StuID	StuName	Dept
S101	Rakib	CSE
S102	Sadia	CSE
S103	Tanvir	EEE
S104	Mim	CSE
S105	Arif	EEE
S106	Nabila	CSE

S106 has no enrolment — used later for EXISTS.

COURSE

CourseID	Title	Dept	Credit
CSE-3101	DBMS	CSE	3.0
CSE-3103	OOP	CSE	3.0
CSE-4105	AI	CSE	3.0
EEE-2101	Circuits	EEE	3.0

ENROLLMENT

StuID	CourseID	Marks
S101	CSE-3101	82
S101	CSE-3103	75
S102	CSE-3101	91
S102	CSE-4105	88
S103	EEE-2101	68
S104	CSE-3101	55
S104	CSE-3103	*NULL*
S105	EEE-2101	77

Facts worth memorising

8 enrolment rows · 7 of them graded · 5 distinct students enrolled · total marks 536 · class average 76.57 · lowest 55 · highest 91.

The NULL rule

Every aggregate except COUNT(*) silently skips NULL. That single row (S104) explains most “wrong” answers today.

AGGREGATE FUNCTION 1 OF 5

COUNT

COUNT(*) counts rows. COUNT(column) counts non-NULL values. COUNT(DISTINCT column) counts distinct non-NULL values.

TYPE AND RUN THESE

```sql
-- 1. how many rows are in the table?
SELECT COUNT(*) AS TotalRows
FROM ENROLLMENT;

-- 2. how many rows actually have a mark?
SELECT COUNT(Marks) AS Graded
FROM ENROLLMENT;

-- 3. how many different students enrolled?
SELECT COUNT(DISTINCT StuID) AS Students
FROM ENROLLMENT;

-- 4. count inside a filter
SELECT COUNT(*) AS CseEnrolments
FROM ENROLLMENT
WHERE CourseID LIKE 'CSE-%';
```

RESULTS

Query	Expression	Result
1	COUNT(*)	**8**
2	COUNT(Marks)	**7**
3	COUNT(DISTINCT StuID)	**5**
4	COUNT(*) + WHERE	**6**

Read the gap

8 − 7 = 1 ungraded row. Subtracting COUNT(*) from COUNT(col) is the fastest way to audit missing data in any table.

AGGREGATE FUNCTIONS 2 AND 3 OF 5

SUM and AVG

Both work on numeric columns only, and both ignore NULL — which means AVG divides by the number of non-NULL values, not by the number of rows.

TYPE AND RUN THESE

```sql
-- 1. total and average in one query
SELECT SUM(Marks)  AS TotalMarks,
       AVG(Marks)  AS AvgMarks
FROM ENROLLMENT;

-- 2. round the average to 2 decimal places
SELECT ROUND(AVG(Marks), 2) AS AvgMarks
FROM ENROLLMENT;

-- 3. treat a missing mark as zero instead
SELECT ROUND(AVG(COALESCE(Marks, 0)), 2) AS AvgZero
FROM ENROLLMENT;

-- 4. average of one course only
SELECT ROUND(AVG(Marks), 2) AS DbmsAvg
FROM ENROLLMENT
WHERE CourseID = 'CSE-3101';
```

RESULTS

Query	Column	Result
1	TotalMarks	**536**
1	AvgMarks	**76.5714**
2	AvgMarks	**76.57**
3	AvgZero	**67.00**
4	DbmsAvg	**76.00**

76.57 vs 67.00

Same column, two different “averages”: 536/7 versus 536/8. Decide which one the question is asking for before you write AVG.

AGGREGATE FUNCTIONS 4 AND 5 OF 5

MIN and MAX

MIN and MAX return the smallest and largest value of a column — and unlike SUM and AVG they also work on text and dates.

TYPE AND RUN THESE

```sql
-- 1. lowest, highest and the spread
SELECT MIN(Marks) AS Lowest,
       MAX(Marks) AS Highest,
       MAX(Marks) - MIN(Marks) AS Spread
FROM ENROLLMENT;

-- 2. MIN / MAX on a text column
SELECT MIN(Title) AS FirstAlphabetically,
       MAX(Title) AS LastAlphabetically
FROM COURSE;

-- 3. all five aggregates in a single pass
SELECT COUNT(*) AS N, SUM(Marks) AS Total,
       ROUND(AVG(Marks),2) AS Average,
       MIN(Marks) AS Lo, MAX(Marks) AS Hi
FROM ENROLLMENT;
```

QUERY 1 AND 2

Lowest	Highest	Spread
55	91	36

FirstAlphabetically	LastAlphabetically
AI	OOP

QUERY 3

N	Total	Average	Lo	Hi
8	536	76.57	55	91

Note the mixed row in query 3: N counts all 8 rows while Total, Average, Lo and Hi are computed from only the 7 graded ones.

ONE RESULT ROW PER GROUP

GROUP BY

GROUP BY splits the rows into buckets and runs the aggregate once per bucket. Every column in the SELECT list must be either grouped or aggregated.

COURSE-WISE SUMMARY

```
SELECT CourseID,
       COUNT(*)            AS Enrolled,
       COUNT(Marks)        AS Graded,
       ROUND(AVG(Marks), 2) AS AvgMarks
FROM ENROLLMENT
GROUP BY CourseID
ORDER BY AvgMarks DESC;
```

RESULT

CourseID	Enrolled	Graded	AvgMarks
CSE-4105	1	1	**88.00**
CSE-3101	3	3	**76.00**
CSE-3103	2	1	**75.00**
EEE-2101	2	2	**72.50**

The classic error

```
SELECT CourseID, StuID, COUNT(*) ... GROUP BY CourseID;
```

StuID is neither grouped nor aggregated — the DBMS rejects the query.

How to read it

CSE-3103 shows Enrolled = 2 but Graded = 1, and its average is 75 rather than 37.5. The bucket has two rows; only one carries a mark.

GROUPING ACROSS TABLES

GROUP BY with JOIN

Real reports group by a column that lives in another table. Join first, then group — and you may group by more than one column.

DEPARTMENT-WISE REPORT

```
SELECT c.Dept,
       COUNT(*)              AS Enrolments,
       ROUND(AVG(e.Marks),2) AS AvgMarks
FROM ENROLLMENT e
JOIN COURSE c ON c.CourseID = e.CourseID
GROUP BY c.Dept;
```

GROUPING BY TWO COLUMNS

```
SELECT c.Dept, e.CourseID, COUNT(*) AS Total
FROM ENROLLMENT e
JOIN COURSE c ON c.CourseID = e.CourseID
GROUP BY c.Dept, e.CourseID
ORDER BY c.Dept, Total DESC;
```

RESULT 1

Dept	Enrolments	AvgMarks
CSE	6	**78.20**
EEE	2	**72.50**

RESULT 2

Dept	CourseID	Total
CSE	CSE-3101	**3**
CSE	CSE-3103	**2**
CSE	CSE-4105	**1**
EEE	EEE-2101	**2**

CSE shows 6 enrolments but its average uses 5 marks — the ungraded row still counts as an enrolment. Always state which number the report means.

FILTERING ROWS VS FILTERING GROUPS

HAVING

WHERE removes rows before grouping and cannot see an aggregate. HAVING removes whole groups after aggregation and is the only place an aggregate may be tested.

RUN BOTH AND COMPARE

```sql
-- WHERE filters ROWS, before grouping
SELECT CourseID, COUNT(*) AS Graded
FROM ENROLLMENT
WHERE Marks IS NOT NULL
GROUP BY CourseID;

-- HAVING filters GROUPS, after aggregation
SELECT CourseID, ROUND(AVG(Marks),2) AS AvgMarks
FROM ENROLLMENT
WHERE Marks IS NOT NULL
GROUP BY CourseID
HAVING AVG(Marks) > 75;
```

WHERE ONLY

CourseID	Graded
CSE-3101	3
CSE-3103	1
CSE-4105	1
EEE-2101	2

WHERE + HAVING

CourseID	AvgMarks
CSE-3101	**76.00**
CSE-4105	**88.00**

ORDER OF EXECUTION

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Because SELECT runs after HAVING, some systems reject HAVING AvgMarks > 75 by alias — repeat the aggregate instead.

Scalar Subqueries

A scalar subquery returns exactly one row and one column, so it can be used anywhere a single value is allowed — in SELECT, in WHERE, even in HAVING.

ONE VALUE, USED THREE WAYS

```sql
-- in the SELECT list and in WHERE
SELECT StuID, CourseID, Marks,
       (SELECT ROUND(AVG(Marks),2)
        FROM ENROLLMENT) AS ClassAvg
FROM ENROLLMENT
WHERE Marks > (SELECT AVG(Marks) FROM ENROLLMENT);

-- in HAVING: courses above the class average
SELECT CourseID, ROUND(AVG(Marks),2) AS AvgMarks
FROM ENROLLMENT
GROUP BY CourseID
HAVING AVG(Marks) > (SELECT AVG(Marks) FROM ENROLLMENT);
```

RESULT 1

StuID	CourseID	Marks	ClassAvg
S101	CSE-3101	**82**	76.57
S102	CSE-3101	**91**	76.57
S102	CSE-4105	**88**	76.57
S105	EEE-2101	**77**	76.57

RESULT 2

CourseID	AvgMarks
CSE-4105	**88.00**

If the inner query returns two or more rows, the whole statement fails. When that happens, you need IN, ANY or ALL instead.

SUBQUERIES 2 OF 5

IN and NOT IN

IN tests membership against a list of values produced by the inner query. The inner query may return many rows — that is exactly the point.

MEMBERSHIP TESTS

```sql
-- 1. who is enrolled in DBMS?
SELECT StuName
FROM STUDENT
WHERE StuID IN (SELECT StuID FROM ENROLLMENT
                WHERE CourseID = 'CSE-3101');

-- 2. who is NOT enrolled in DBMS?
SELECT StuName
FROM STUDENT
WHERE StuID NOT IN (SELECT StuID FROM ENROLLMENT
                    WHERE CourseID = 'CSE-3101');

-- 3. IN also accepts a plain literal list
SELECT * FROM COURSE WHERE Dept IN ('CSE','EEE');
```

RESULT 1

StuName
Rakib
Sadia
Mim

RESULT 2

StuName
Tanvir
Arif
Nabila

NOT IN trap: if the inner list contains a single NULL, the whole condition is unknown and you get zero rows back.

SUBQUERIES 3 OF 5

ANY and ALL

ANY is true if the comparison holds for at least one value of the inner set; ALL is true only if it holds for every value. Both need a comparison operator in front.

COMPARE AGAINST A WHOLE SET

```
-- inner set = the EEE marks: 68 and 77

-- 1. beats AT LEAST ONE of them  (> 68)
SELECT StuID, CourseID, Marks FROM ENROLLMENT
WHERE Marks > ANY (SELECT Marks FROM ENROLLMENT
                   WHERE CourseID = 'EEE-2101');

-- 2. beats EVERY one of them  (> 77)
SELECT StuID, CourseID, Marks FROM ENROLLMENT
WHERE Marks > ALL (SELECT Marks FROM ENROLLMENT
                   WHERE CourseID = 'EEE-2101');
```

RESULT 1 · > ANY

StuID	CourseID	Marks
S101	CSE-3101	**82**
S101	CSE-3103	**75**
S102	CSE-3101	**91**
S102	CSE-4105	**88**
S105	EEE-2101	**77**

RESULT 2 · > ALL

StuID	CourseID	Marks
S101	CSE-3101	**82**
S102	CSE-3101	**91**
S102	CSE-4105	**88**

> ANY ≡ > MIN(set) **> ALL ≡ > MAX(set)**

SUBQUERIES 4 OF 5

EXISTS and NOT EXISTS

EXISTS does not care what the inner query selects — only whether it returns at least one row. It is the safe way to ask “is there a matching record?”

PRESENCE AND ABSENCE

```
-- 1. students who have enrolled in something
SELECT s.StuID, s.StuName
FROM STUDENT s
WHERE EXISTS (SELECT 1 FROM ENROLLMENT e
              WHERE e.StuID = s.StuID);

-- 2. students who have enrolled in nothing
SELECT s.StuID, s.StuName
FROM STUDENT s
WHERE NOT EXISTS (SELECT 1 FROM ENROLLMENT e
                  WHERE e.StuID = s.StuID);
```

SELECT 1 is a convention: the engine stops at the first matching row, so the column list is never used.

RESULT 1 · EXISTS

StuID	StuName
S101	Rakib
S102	Sadia
S103	Tanvir
S104	Mim
S105	Arif

RESULT 2 · NOT EXISTS

StuID	StuName
S106	**Nabila**

Prefer NOT EXISTS over NOT IN whenever the inner column can be NULL — EXISTS has no NULL trap.

Correlated Subqueries

A correlated subquery references a column from the outer query, so it cannot run on its own — it is re-evaluated once for every outer row.

COMPARE EACH ROW WITH ITS OWN GROUP

```
-- marks above the average of THAT SAME course
SELECT e.StuID, e.CourseID, e.Marks
FROM ENROLLMENT e
WHERE e.Marks > (SELECT AVG(e2.Marks)
                 FROM ENROLLMENT e2
                 WHERE e2.CourseID = e.CourseID);
```

CORRELATED SUBQUERY IN THE SELECT LIST

```
SELECT s.StuName,
       (SELECT COUNT(*) FROM ENROLLMENT e
        WHERE e.StuID = s.StuID) AS Courses
FROM STUDENT s;
```

RESULT 1

StuID	CourseID	Marks
S101	CSE-3101	**82**
S102	CSE-3101	**91**
S105	EEE-2101	**77**

RESULT 2

StuName	Courses
Rakib	**2**
Sadia	**2**
Tanvir	**1**
Mim	**2**
Arif	**1**
Nabila	**0**

Try running the inner query alone: it fails, because e.CourseID has no meaning outside.

TO BE SOLVED AND SUBMITTED

Lab Tasks

Write one query per task on the same three tables. Verify each answer by hand against the data on slide 2 before you accept it.

1. Count how many courses each department offers.
2. Show the highest mark of every course together with the course title.
3. List the courses in which more than two students are enrolled.
4. Show every student with their average mark, including students with none.
5. Find the students whose average mark is above the class average.
6. List the courses in which nobody has scored above 80 (use NOT EXISTS).
7. Rewrite task 5 using IN, then using EXISTS, and compare the results.
8. Find the second-highest mark in ENROLLMENT without using LIMIT.

Submission

One .sql file named <StudentID>_Lab_Aggregation.sql — number every answer in a comment and paste the output below each query.